

A Re-Architected Development Lifecycle

The AI-Native SDLC

Version 2.0

Team-Level Operations Under Sealed Enterprise Architecture

Version 2.0 · April 2026 · Suneesh Sidharthan

What this document is

The AI-Native SDLC v2.0 is a re-architected development lifecycle for teams using AI assistants to build software within organisations that have adopted Sealed Enterprise Architecture (SEA) as their foundational architectural philosophy.

This is a Major Version Bump. Version 1.1 — the previous SDLC — is archived with its full history preserved. Version 2.0 supersedes it as the current AI-Native SDLC, built on SEA-native foundations rather than the informal sealed architecture that preceded SEA's formal articulation.

Two modes are specified. Full mode assumes SEA governance is operational at the enterprise level. Team-autonomous mode supports teams that want SDLC v2.0's discipline without requiring organisational SEA adoption first. Both produce SEA-compatible artefacts; the difference is where the governance authority resides.

Contents

§	Section	Description
	FRONT MATTER	
	Preface	Why v2.0, what changed, version relationship to v1.1
	How to Read This Document	Three reader paths
	PART I — FOUNDATION	
1	Why an AI-Native SDLC Needs a SEA Foundation	The premise shift that warranted a Major Version Bump
2	Full Mode and Team-Autonomous Mode	Two ways to adopt v2.0
3	What Transfers from v1.1, What Is New	Honest inventory of the re-architecture
	PART II — THE SEVEN PHASES	
4	Phase 1 — FRAME	Problem, core bet, contract candidates
5	Phase 2 — SEAL	Full SEA sealing protocol at team level
6	Phase 3 — SPEC	Extension proposals, verification requirements, consumer mapping
7	Phase 4 — BUILD	Implementation against sealed contracts
8	Phase 5 — VERIFY	Five compliance states and severity classification
9	Phase 6 — RECONCILE	Alignment with SEA quarterly governance
10	Phase 7 — EVOLVE	Extension, parallel-live, major version bump at team level
	PART III — ARTEFACTS	
11	The Contract and Intent Pair	Successor to the v1.1 SSOT
12	The Change History	Successor to the v1.1 Decision Log for contract matters
13	The Extension Proposal	Successor to the v1.1 Feature Checklist
14	The Drift Incident Log	Team-level log feeding enterprise governance
	PART IV — TEAM-TO-ENTERPRISE INTERFACE	
15	Proposing Contracts Upward	How teams surface contract candidates to the SRB
16	Extension Reviews That Span Scales	Team drafts, SRB approves

§	Section	Description
17	The RECONCILE → Quarterly Review Bridge	How team rhythms connect to enterprise rhythms
18	Operating in Team-Autonomous Mode	When enterprise SEA is not yet adopted
	PART V — MIGRATION FROM v1.1	
19	Who Should Migrate, Who Should Not	Honest guidance on staying on v1.1
20	The Migration Path	Step-by-step, typical timeline
	ANNEXES	
A	CD1–CD4 Reframed Through SEA	What v1.1 called drift incidents, v2.0 calls governance gaps
B	Terminology Mapping v1.1 → v2.0	Every changed term, with rationale
C	Version Relationship Record	How v2.0 relates to v1.1 under SEA retirement discipline

Preface

The AI-Native SDLC v1.1 was written in early 2026, during a production build of a single platform by a solo engineer working with multiple generations of AI assistants. The framework it described was derived from the specific governance patterns that made that build survivable: one chat per unit of work, source-first enforcement, cross-family AI review, and a seven-phase lifecycle that made space for the reconciliation sweeps the build kept needing.

Version 1.1 was also written before Sealed Enterprise Architecture (SEA) had been formally articulated. At the time, the SDLC referenced 'sealed interfaces' and 'anti-drift-by-design' as principles without being able to point to the foundational framework those principles implied. SEA 1.0, published later in the same month, filled in that foundation. It articulated the philosophy that the SDLC had been quietly resting on: that long-lived systems must explicitly separate the durable from the transient, and that sealed contracts are the durable layer.

With SEA now published, a decision presented itself. The SDLC could remain at v1.1 with an integration guide pointing to SEA. It could be lightly extended to v1.2 with SEA references added. Or it could be properly re-architected as v2.0, treating SEA as its foundation rather than as a separately-articulated companion.

This document is the v2.0 option. It was not written because v1.1 was failing — v1.1 serves its audience, and teams currently using it should continue doing so until they choose to migrate. It was written because the premises that produced v1.1 have been superseded. Version 1.1 assumed the SDLC was a standalone framework; v2.0 operates as a team-level layer within SEA governance. Version 1.1 described artefacts informally (SSOT, Decision Log, Feature Checklist); v2.0 uses SEA-native artefacts (sealed contract with paired intent document, change history, extension proposals). Version 1.1 defined sealing as a matter of team discipline; v2.0 defines it in the full SEA sense, with the governance, verification, and evolution protocols that implies.

The Major Version Bump is performed in the same discipline SEA specifies for any sealed artefact: v1.1 is archived, not deleted. Its specification remains readable. Its history is preserved. Teams currently using v1.1 are not obligated to migrate; they can continue operating on v1.1 until the point at which v2.0's additional capabilities become valuable enough to justify the transition. The bridge analogy in SEA 1.0 applies here too — the old framework stays in service alongside the new one until consumers have migrated at their own cadence.

This is a Major Version Bump, not a patch. The SDLC is not better in v2.0; it is differently shaped, for a different set of assumptions about the world around it.

One honest note on scope. Version 2.0 is more ambitious than v1.1. It assumes more about the organisations adopting it, particularly in its full mode. Teams working in environments without SEA adoption at enterprise level can still use v2.0 in team-autonomous mode, which is described fully in Part IV — but the natural home of this framework is an organisation that has adopted SEA and wants a coherent way for its teams to operate within SEA governance. Teams in that situation will find v2.0 substantially more useful than v1.1. Teams in simpler environments may find v1.1 continues to serve them better, and that is a legitimate choice.

Two companion documents are referenced throughout. Sealed Enterprise Architecture 1.0 — the foundational whitepaper — establishes the philosophy and principles this SDLC rests on. The SEA Implementation Handbook, Volume 1 — the operational companion to SEA 1.0 — specifies the measurement, enforcement, verification, and evolution mechanics at the enterprise level. This SDLC

focuses on the team level, and assumes readers will consult those documents when deeper detail is required.

How to Read This Document

Three paths through the document, depending on the reader's role and situation:

If you are a tech lead on a team that has adopted v1.1

Read the Preface, then Part V (Migration). The migration guidance will tell you whether v2.0 is worth the transition for your team right now, or whether staying on v1.1 for another cycle is the right choice. If you decide to migrate, Parts II and III become your operational reference. Part I is context that matters less urgently.

If you are a platform architect adopting v2.0 for a new team

Read in order. Part I establishes the foundation. Part II walks through the seven phases with their SEA-native artefacts and gates. Part III details the artefacts themselves. Part IV is essential if you are operating in an enterprise that has adopted SEA — it specifies how the team connects to enterprise governance. Part V is skippable if you are not coming from v1.1.

If you are an enterprise architect integrating the team-level SDLC into SEA governance

Read the Preface, then Parts I and IV. The foundation (Part I) establishes the premise that this SDLC operates as a team-level layer within SEA. Part IV specifies the team-to-enterprise interface — how contracts flow upward, how extension reviews span scales, how team rhythms connect to SRB rhythms. Parts II, III, and V are operational details most relevant to the teams, not to enterprise-level integration.

A note on companion documents

This SDLC is intentionally more compact than SEA 1.0 and the Implementation Handbook. It does not repeat their content. When a phase references sealing, the full sealing protocol lives in SEA 1.0. When a gate references compliance states, the full compliance state specification lives in the Handbook.

Readers who do not have access to, or interest in, the companion documents can follow the SDLC narratively — the summary treatments here are sufficient for orientation. Readers who will actually implement the framework should have both companion documents available for reference.

PART I

Foundation

Why the SDLC needs a SEA foundation. How v2.0 relates to v1.1. Full mode and team-autonomous mode.

1. Why an AI-Native SDLC Needs a SEA Foundation

The AI-Native SDLC v1.1 addressed a specific problem: how does a team using AI assistants avoid the silent drift that accumulates across sessions, model generations, and rapid iteration? Its answer was process — phases, gates, rituals, artefacts that keep the team aligned with itself over time.

That answer works at team scale. It begins to strain at enterprise scale, for reasons the v1.1 framework was not designed to address.

1.1 The Scale Problem

A team of five to fifteen engineers working with AI assistants on a single product can hold the system in its head, use informal SSOT documents, rely on personal familiarity to catch drift, and operate within the team's own governance. The v1.1 framework assumed this scale and built for it.

An enterprise of several hundred engineers operating dozens of services under long-term governance cannot rely on personal familiarity. Its SSOT cannot be a shared Google Doc. Its drift catches cannot rely on any single engineer recognising the pattern. Its governance cannot be ad-hoc peer review. It needs formal sealed contracts with independent owners, cryptographic enforcement, quarterly governance cadences, and the succession protocols that make architecture survive beyond any individual's tenure.

This is the gap SEA fills. The question for the SDLC was whether to bridge that gap through a separate integration document — which would leave two frameworks operating side by side — or to re-architect the SDLC itself so that it naturally scales from team to enterprise by sitting inside SEA governance. Version 2.0 takes the latter approach.

1.2 The Premise Shift

Five premises changed between v1.1 and v2.0. Each is worth naming explicitly, because the re-architecture makes sense only in light of the shift.

Premise	v1.1	v2.0
Primary unit of concern	The team working with AI assistants	The enterprise operating long-lived systems, within which teams operate
Artefact formality	Informal (SSOT, Decision Log, Feature Checklist shaped by what worked)	SEA-native (sealed contracts, intent documents, governance records)
Governance model	Lightweight; mostly team-internal with 'Ready for external review' as the main gate	Layered; team-level protocols feed into enterprise SRB
Framework independence	Self-contained standalone framework	Team-level operational layer within SEA's foundational philosophy
Meaning of 'sealed'	Informally protected by discipline and peer review	Full SEA definition — durable, formally protected, cryptographically enforced

1.3 What This Implies For the Phases

The seven phases of the SDLC — FRAME, SEAL, SPEC, BUILD, VERIFY, RECONCILE, EVOLVE — remain. Their names remain. The work each phase governs remains broadly the same. But the specifics change substantially in v2.0:

- FRAME now explicitly produces contract candidates, not just problem statements
- SEAL is now the full SEA sealing protocol, including intent document production, not informal interface declaration
- SPEC now produces extension proposals and verification requirements, not informal specification documents
- BUILD is now conducted against sealed contracts with Source-First enforcement, with implementations understood as transient
- VERIFY now maps to SEA's five compliance states and severity classification, not just L1/L2/L3 tiers
- RECONCILE now aligns with SEA's quarterly governance review and feeds into the enterprise drift log, rather than operating purely at team scale
- EVOLVE now implements SEA's three evolution mechanisms — extension, parallel-live replacement, Major Version Bump — at team level

The phases still feel familiar to a v1.1 practitioner. A team migrating from v1.1 to v2.0 does not need to learn a new vocabulary of phases. What they need to learn is that each phase now produces SEA-shaped artefacts and connects to SEA-shaped governance, rather than producing informal artefacts that served a standalone framework.

The phases stay familiar because the work stays familiar. What changes is what each phase produces, and where what it produces is governed.

2. Full Mode and Team-Autonomous Mode

The SDLC v2.0 operates in one of two modes, selected at adoption time based on the organisational context in which the team is working. Both modes produce SEA-compatible artefacts. The difference is where the governance authority resides and how external the review is.

2.1 Full Mode

Full mode assumes the organisation has adopted Sealed Enterprise Architecture. This means an operating Sealed Review Board exists, formal Contract Owner and Reviewer roles are recognised, the enterprise drift incident log is active, and quarterly governance cadences are in place.

In full mode, the team's SDLC operation connects directly to the enterprise governance structure:

- Contract candidates produced in FRAME are proposed to the SRB through the team's architect
- Contracts are sealed with approval from the governance body that has jurisdiction, not just from the team
- Extension proposals produced in SPEC are reviewed by the Contract Owner and Reviewer, who may be outside the team
- VERIFY gates include the enterprise verifier, running against the enterprise contract registry
- RECONCILE sweeps feed into quarterly SRB reviews
- EVOLVE operations that cross team boundaries are coordinated through the SRB

Full mode is the natural home of v2.0. It is where the framework produces its greatest value, because the team's disciplined process feeds into an enterprise governance structure that amplifies and protects that discipline across the organisation's lifetime.

2.2 Team-Autonomous Mode

Team-autonomous mode exists because many teams work in organisations that have not adopted SEA at enterprise level. These teams may still want SDLC v2.0's discipline — SEA-shaped artefacts, proper sealing protocols, the improved compliance vocabulary — without waiting for their organisation to commit to SEA as a whole.

In team-autonomous mode, the team operates SDLC v2.0 with all SEA-native artefacts, but the governance authorities become team-internal:

- The team itself acts as its own Sealed Review Board, meeting on a quarterly cadence
- Contract Owner and Reviewer roles exist within the team, ideally held by different people who rotate
- The drift incident log is team-local, not tied to enterprise governance
- Verification runs against team-local tooling, not enterprise-level verifiers
- Major Version Bumps are approved by the team's own leadership with senior technical stakeholder involvement

Team-autonomous mode is not a watered-down version of full mode. The same artefacts are produced, the same discipline is exercised, the same compliance states and severity classifications apply. What changes is that the governance authority does not extend beyond the team. If the organisation later adopts SEA, the team's sealed contracts and intent documents are ready to be brought under enterprise governance without rework — the artefacts were SEA-shaped from the start.

2.3 When to Choose Which

Situation	Recommended mode
Organisation has adopted SEA; SRB is operational; governance charter is in place	Full mode
Organisation is in SEA pilot stage (Section 12.1 of SEA 1.0), and your team is the pilot	Full mode, with the caveat that some enterprise protocols are still being established
Organisation has not adopted SEA, but the team wants the discipline	Team-autonomous mode
Team is operating inside an organisation that explicitly rejects formal architecture governance	Team-autonomous mode, with realistic expectations about how far the discipline can travel
Team is small (under 5 engineers) and early-stage	Either mode, or consider staying on SDLC v1.1 for simplicity

2.4 Bridging from Team-Autonomous to Full

A team that begins in team-autonomous mode can transition to full mode when its organisation adopts SEA. The transition is generally straightforward because the artefacts are compatible:

1. The team's sealed contracts are presented to the organisation's Sealed Review Board for ratification into the enterprise portfolio
2. Team-local Contract Owners are either confirmed in enterprise roles or succeeded by enterprise-designated owners
3. Team-local drift incidents are migrated into the enterprise log with full history preserved
4. Team verifier configurations are adapted to run against enterprise-level tooling
5. The team's quarterly review rhythm aligns to the enterprise SRB cadence

The transition typically takes one to two quarterly cycles. Organisations adopting SEA often find that a team that had been operating in autonomous mode is among the easiest to bring into full mode, because the team already operates on SEA-compatible artefacts. This is a reason to recommend team-autonomous mode even when enterprise SEA adoption is plausible but not yet committed.

3. What Transfers from v1.1, What Is New

A Major Version Bump does not start from zero. Much of what v1.1 captured was correct and remains correct in v2.0. This section provides an honest inventory of what transfers, what changes, and what is genuinely new, so that teams migrating have a clear sense of the scope of the transition.

3.1 What Transfers Unchanged

These elements of v1.1 appear in v2.0 substantially unchanged because they were right at the time and remain right:

- The seven-phase structure: FRAME → SEAL → SPEC → BUILD → VERIFY → RECONCILE → EVOLVE
- The 'Ready for external review' load-bearing phrase as the primary BUILD → VERIFY gate
- The one-chat-per-unit-of-work rule for AI-assisted development
- The Source-First rule: view the file before claiming anything about it
- The cross-family AI review principle: the reviewer should be a different model family from the builder
- The model-agnostic-by-default principle: phases and gates do not depend on any specific AI model
- The RACI with AI-Builder and AI-Reviewer as distinct roles
- The two-level VERIFY structure (Full vs Lite) for different kinds of change

A team comfortable with these v1.1 concepts will find them still operative in v2.0. What changes is what they produce and how they connect to broader governance.

3.2 What Changes

These elements of v1.1 appear in v2.0 in modified form:

v1.1 element	v2.0 treatment
SSOT (Single Source of Truth document)	Replaced by the sealed contract + paired intent document pair. The SSOT was an informal precursor; the contract+intent pair is its SEA-native realisation.
Decision Log	For contract-related decisions, replaced by the change history within the sealed contract. Non-contract decisions still use a team decision log, but it is no longer the primary artefact.
Feature Checklist Template	Replaced by the extension proposal template when the feature involves a sealed contract. Informal features continue to use the team's ordinary issue tracking.
L1 / L2 / L3 VERIFY tiers	Replaced by the five compliance states (Conforming, Conforming-with-Warnings, Partial, Non-Conforming, Unverifiable) with severity classification. Richer vocabulary; richer decision guidance.
Drift Incident Log (CD-series)	Retained but re-contextualised. In full mode, team incidents feed into the enterprise log. In team-autonomous mode, the log remains team-local but follows SEA's severity classification.
Reconciliation Pair A-E sweeps	Retained in RECONCILE phase but now also drive inputs into the

v1.1 element	v2.0 treatment
	SRB's quarterly review in full mode.
'Drift vs Evolution' principle	Retained but now formalised through SEA's three evolution mechanisms rather than left as a philosophical distinction.

3.3 What Is New in v2.0

These elements did not exist in v1.1 and are introduced in v2.0:

- Full mode vs team-autonomous mode distinction — explicit operating contexts
- Sealed contract production as a first-class phase output (SEAL phase)
- Intent document as a required artefact paired with every sealed contract
- Extension proposal as a formal artefact type with its own review protocol
- Compliance state reporting from the verifier, replacing binary pass/fail
- Team-to-SRB contract proposal protocol (full mode only)
- Cross-scale extension review (team drafts, SRB approves in full mode)
- Succession protocol for Contract Owner roles within the team
- Explicit parallel-live replacement playbook at team scale
- Team-level Major Version Bump procedure

None of these are complex in isolation. Their combined effect is to give team-level work a clear line of sight to enterprise governance, which is the fundamental shift v2.0 achieves.

3.4 What Is Removed

Very little is genuinely removed. One clarification: v1.1's informal references to 'sealed modules' that were protected only by team discipline are subsumed by v2.0's full SEA sealing protocol. A team migrating from v1.1 that had practised informal module sealing will find v2.0 provides a more robust and more widely-recognised version of the same intuition.

The migration is substantive but learnable

A team moving from v1.1 to v2.0 will find that approximately 60% of its habits transfer unchanged, 30% are refinements of existing practice, and 10% are genuinely new.

The 10% that is new — sealed contract production, intent documents, extension proposals, compliance states, enterprise interface — is where the migration effort concentrates. Part V specifies a typical one-to-two-sprint transition timeline.

Teams currently on v1.1 should not migrate for novelty. They should migrate when the additional capabilities become valuable — typically when the team scales past 15 engineers, when the organisation adopts SEA, or when the product enters a phase where formal governance becomes a commercial or regulatory requirement.

PART II

The Seven Phases

*FRAME · SEAL · SPEC · BUILD · VERIFY · RECONCILE · EVOLVE Re-architected
around SEA-native artefacts and governance.*

The Phases at a Glance

Before the detailed treatment of each phase, a summary view of all seven. Each phase has a trigger that initiates it, a primary AI role during execution, the SEA-native artefacts it produces, the exit gate that moves it to the next phase, and the SEA governance touchpoint in full mode.

Phase	Trigger	Primary output	Exit gate
FRAME	New product or feature requested	Problem statement + contract candidates	Core bet statable in 90 seconds
SEAL	FRAME locked	Sealed contracts + intent documents	Contracts reviewed and approved
SPEC	SEAL approved	Extension proposals + verification requirements	All specs pass external review
BUILD	SPEC locked	Implementations conforming to contracts	'Ready for external review' declared
VERIFY	BUILD declares itself done	Compliance state report per contract	All states Conforming or Conforming-with-Warnings
RECONCILE	End of sprint / week / milestone	Drift incident log updated	Zero open deltas
EVOLVE	New capability needed	Extension / parallel-live / MVB proposal	Evolution mechanism approved and executed

The phases are sequential for any single unit of work, but the overall pipeline allows multiple units in different phases concurrently. A team working on three parallel features might have one in FRAME, one in BUILD, and one in VERIFY, with RECONCILE happening on a weekly rhythm across all of them. EVOLVE is event-driven rather than sequential — it is invoked when a change requires it, regardless of which phase other work is in.

4. Phase 1 — FRAME

FRAME is the first and often the most neglected phase. Its purpose is to establish what is being built before any code is written and, critically in v2.0, before any sealed contracts are proposed. The discipline of FRAME is saying no to solutions until the problem is understood.

4.1 Trigger

FRAME begins when someone — a product manager, an engineering leader, a founder, or the team itself — proposes that a new product, feature, module, or service should exist. The proposal may be a single sentence or a ten-page document. Either way, FRAME treats it as raw input requiring distillation, not as specification.

4.2 Primary AI Role

The AI's role in FRAME is Socratic, not productive. The AI should ask questions, reflect back assumptions, surface contradictions, and help the team articulate what is genuinely being proposed. The AI should resist the urge to produce solutions. A team that lets its AI assistant jump to solutions in FRAME has skipped FRAME.

This is a stylistic discipline the team must enforce with explicit prompting. The recommended pattern: begin FRAME conversations with a prompt that instructs the AI to help the team arrive at clarity, not to propose implementations. When the AI begins drifting toward solutions, the team redirects.

4.3 Outputs

FRAME in v2.0 produces five outputs. The first four appeared in v1.1; the fifth is new and is the key bridge into SEAL:

6. Problem Statement — one paragraph, plain language, describing the problem to be solved
7. User and Context Brief — who, what device, what constraints, what environment
8. Core Bet — the testable thesis this work assumes to be true
9. Non-Goals — explicit list of what this work is not intended to accomplish
10. Contract Candidates — initial identification of durable boundaries that may warrant sealing

The Contract Candidates output is v2.0's most significant addition to FRAME. Before any design or code, the team asks: what in this problem space is durable enough to warrant a sealed contract? What interfaces will we want to still hold in three years? What invariants are so central to the domain that violating them would mean we had built something different?

The candidates identified in FRAME are not yet sealed. They are proposals. They will be examined, refined, and either sealed or set aside in the SEAL phase. But surfacing candidates during FRAME — before solutions are proposed — prevents a common failure mode: sealing contracts around the first implementation that comes to mind rather than around the durable boundaries of the domain itself.

4.4 Exit Gate

The exit gate for FRAME is identical to v1.1's: the team can state the Problem, the Core Bet, and the Success Metric out loud in under 90 seconds without referring to any document. This test is harder than it sounds. Teams that cannot pass the test usually have not distilled the problem far enough — they have a rough sense but not a precise one.

In addition, v2.0 adds a second gate element: the team has identified at least one contract candidate and articulated why it is a candidate. A FRAME phase that ends without any contract candidate

proposed is either a phase where nothing durable is being built (legitimate — sometimes work is genuinely transient) or a phase where durable boundaries are being missed (a warning sign for SEAL).

4.5 Full Mode vs Team-Autonomous Mode

In full mode, contract candidates identified in FRAME are shared with the team's architect liaison to the SRB during the phase rather than after. This early conversation lets the architect flag candidates that may overlap with existing sealed contracts in the enterprise, suggest candidates the team has not considered, and prevent the team from investing deeply in sealing something the enterprise has already sealed differently elsewhere.

In team-autonomous mode, the candidate identification remains team-internal. The team's senior architect or tech lead plays the role the enterprise architect would play in full mode.

5. Phase 2 — SEAL

SEAL is the most substantively changed phase between v1.1 and v2.0. In v1.1, SEAL was an informal phase where interfaces were declared and anti-drift principles were established. In v2.0, SEAL is the full SEA sealing protocol: each selected contract candidate is developed into a sealed contract with a paired intent document, then formally sealed through the appropriate governance authority.

5.1 Trigger

SEAL begins when FRAME is locked — the Problem, Core Bet, Success Metric, and Contract Candidates are all in place, and the team has exit-gated FRAME. No architecture or code precedes SEAL in v2.0.

5.2 Primary AI Role

In SEAL, the AI acts as a contract drafter. The AI proposes contract shapes, articulates invariants, documents rejected alternatives, and drafts intent documents. The human role is to verify the proposed contracts match the team's actual understanding of what must be durable, and to identify drafts where the AI has over-specified (sealing what should be transient) or under-specified (leaving what should be durable unprotected).

The Source-First rule applies strongly in SEAL. When drafting a contract for an existing system's interface, the AI must view the actual source code, not describe it from memory. Contracts drafted from AI hallucination rather than from actual code are the single most common source of CD-series drift incidents in the case history behind v1.1.

5.3 Outputs

SEAL produces four outputs, all of which conform to the SEA 1.0 specification:

11. One sealed contract YAML per accepted candidate, following the canonical schema specified in SEA 1.0 Section 10
12. One intent document per sealed contract, following the Markdown structure specified in SEA 1.0 Section 9.3
13. Verification strategy for each contract — which invariants will be checked statically, which at runtime, and which require manual review
14. Governance registration — the contract is added to the registry, its Owner and Reviewer are named, and its governance body is recorded

Each contract output is complete, not a stub. An incomplete intent document — one without rejected alternatives, or without a failure modes section, or without rationales on each invariant — is not yet ready for sealing. The SEAL phase includes the work of writing these sections properly, not just the work of declaring the contract's interface.

5.4 Exit Gate

The SEAL exit gate in v2.0:

- Each sealed contract has a complete intent document (all required sections present)
- Each sealed contract has a verification strategy identified
- External review has been completed on each contract (full mode: by the governance body; team-autonomous mode: by a team member who was not the drafter, ideally using a different AI model family)

- The contracts are registered in whatever registry the organisation uses (full mode: enterprise registry; team-autonomous mode: team-local registry)

SEAL ends with the contracts in sealed status. They are now durable — protected by the governance protocol, no longer modifiable through ordinary development work, changeable only through the evolution mechanisms in Phase 7.

5.5 Common Failure Modes in SEAL

Four failure modes recur during SEAL and are worth naming so teams recognise them:

Failure mode	Symptom	Counter
Sealing too much	Contracts are proposed for transient concerns that should be implementation details	Apply the Seal Test: would the system be a different system if this element changed? If no, it is transient.
Sealing too little	Durable boundaries are left unsealed because they do not yet feel important	Look at the FRAME contract candidates; if any are rejected, document why
Rushing the intent document	Invariants are listed without rationale; rejected alternatives section is empty	The intent document is not optional; SEAL is not complete without it
Skipping external review	The team seals contracts without any outside perspective	External review is a hard gate, not a courtesy

6. Phase 3 — SPEC

SPEC is the specification phase. With sealed contracts in place, SPEC produces the detailed specifications needed to implement against those contracts: extension proposals for any extensions to existing contracts, verification requirements detailing how each contract will be checked, and consumer mapping identifying who depends on each contract.

6.1 Trigger

SPEC begins when SEAL is approved. The contracts are sealed. Now the team specifies the work that will implement them.

6.2 Primary AI Role

In SPEC, the AI is a specification author and internal reviewer. It produces the detailed specifications required by the sealed contracts, and it checks those specifications for internal consistency. A common pattern: the AI drafts, a second AI session (ideally in a different model family) reviews, and the human adjudicates the differences.

6.3 Outputs

SPEC produces four outputs:

15. Extension Proposals — for any extensions to existing sealed contracts, formal extension proposals following the template specified in the SEA Implementation Handbook
16. Verification Requirements — per sealed contract, the specific checks that will run statically, at runtime, and through manual review
17. Consumer Mapping — identification of known consumers of each sealed contract, both internal and external, with their current version and migration status
18. Implementation Plan — the sequence of work the team will undertake during BUILD, organised around contracts rather than around features

One important observation about the Implementation Plan: in v2.0, work is organised around contracts, not around user stories or features. A feature that touches three sealed contracts becomes three implementation tasks, each tied to its contract. This organisation is necessary because contracts are the unit of governance; organising work any other way makes it harder to verify that each piece of work conforms to its applicable contract.

6.4 Exit Gate

SPEC exits when:

- All extension proposals have been approved (full mode: by Contract Owners and Reviewers; team-autonomous mode: by the team's own governance)
- Verification requirements are documented for each contract and reviewable
- Consumer mapping is complete and validated against the actual consumer registry
- The Implementation Plan has been reviewed externally and approved
- No TODOs remain in sealed contract specifications

The last point matters. A contract with unfinished specification is not yet sealed; a contract with unfinished SPEC work cannot move to BUILD. 'We will figure this out in BUILD' is not an acceptable answer for anything involving a sealed contract. If it is genuinely unknown, that is SEAL work, not BUILD work.

7. Phase 4 — BUILD

BUILD is implementation. With sealed contracts in place and specifications complete, the team writes the code that implements the contracts. In v2.0, BUILD is conducted against the sealed contracts as specifications; the contracts are the target, not the code.

7.1 Trigger

BUILD begins when SPEC is locked and the Implementation Plan is approved.

7.2 Primary AI Role

In BUILD, the AI is an implementation partner. It writes code that implements the sealed contracts. The human role is to verify that the code actually conforms to the contracts rather than to what the AI remembered about the contracts, and to enforce the Source-First rule when the AI needs to reference existing code.

The one-chat-per-unit rule applies strongly in BUILD. Each atomic unit of implementation work — one sealed contract's implementation, or one extension's implementation — happens in its own chat. The chat does not carry state across units. This prevents cross-unit context pollution and keeps each implementation focused on its own contract.

7.3 Operational Rules During BUILD

Four rules operate continuously during BUILD:

19. Source-First — before claiming anything about an existing file, view it. Never describe what you think the file contains; open it and see.
20. Contract-First — the contract specification is the authority. If the code disagrees with the contract, the code is wrong (unless the contract itself is discovered to be mis-specified, which is an EVOLVE event, not a BUILD event).
21. One chat, one unit — do not let BUILD chats accumulate context across units. Start a fresh chat for each implementation task.
22. Right model for the task — BUILD uses the model appropriate for the work. Deep reasoning for complex algorithms; faster models for routine scaffolding. The Model Selection Matrix in the Master Build Guide specifies the mapping.

7.4 Outputs

BUILD produces:

- Implementation code conforming to each sealed contract
- Tests that exercise each contract's invariants (satisfying the verification requirements from SPEC)
- Updated documentation for transient-layer concerns (implementation-specific documentation; contract-level documentation lives in the intent document and does not change)
- Any runtime instrumentation needed to support the contract's runtime invariants

7.5 The Exit Gate

BUILD exits with the load-bearing phrase inherited from v1.1: 'Ready for external review.' This is not a casual statement. It is a formal declaration by the AI-Builder (and the human engineer pairing with it) that the implementation is complete, that it conforms to the contract, and that it is ready to be examined by an independent reviewer.

Along with the phrase, BUILD produces a handoff package for VERIFY:

- The complete set of changed files
- A review prompt structured for the VERIFY session
- The specific contracts being verified against
- Any known limitations or areas requiring particular attention

A BUILD that says 'Ready for external review' without the handoff package is not actually ready. The phrase is a commitment that the review package exists.

8. Phase 5 — VERIFY

VERIFY is the verification phase. A separate chat — ideally in a different AI model family from the builder — examines the implementation against the sealed contracts and produces a compliance report. In v2.0, the report uses SEA's five compliance states and severity classification rather than v1.1's L1/L2/L3 tiers.

8.1 Trigger

VERIFY begins when BUILD declares 'Ready for external review' and provides the handoff package.

8.2 Primary AI Role

The primary AI in VERIFY is a different AI from the one that built the code. Ideally in a different model family — Claude reviews ChatGPT-built code, Gemini reviews Claude-built code. This cross-family structure is not cosmetic; same-family review systematically misses the failure modes the original model was predisposed to produce.

The VERIFY AI operates in three layers:

23. L1 — Machine — running the verifier against the sealed contract, checking static conformance, SEAL hash verification, lint, typecheck, and any automated assertion of invariants
24. L2 — Second AI — examining whether the code actually does what the sealed contract and intent document specify, catching hallucinated APIs, incorrect assumptions, and conceptual errors
25. L3 — Human — final sanity check before VERIFY declares the unit done

8.3 The Compliance State Report

Unlike v1.1's L1/L2/L3 pass/fail pattern, v2.0's VERIFY produces a compliance state report for each contract-implementation pair. The state is one of the five specified in the SEA Implementation Handbook:

State	Gate behaviour	Next action
Conforming	Passes	Unit is done; merge to main
Conforming with Warnings	Passes with documented warnings	Address warnings in next regular cycle
Partial	Blocks merge	Return to BUILD; specific issues documented
Non-Conforming	Blocks merge; serious concern	Escalate to contract-level review
Unverifiable	Blocks merge; infrastructure concern	Verifier or contract investigation; not an implementation issue

8.4 Severity Classification

Within Partial and Non-Conforming states, individual violations carry severity. Severity classification follows the SEA Implementation Handbook specification (Section 14): Critical, High, Medium, Low.

Critical violations block deployment absolutely. High violations block deployment until addressed. Medium violations permit deployment with warnings. Low violations are cosmetic.

Severity is not judged at review time by the reviewer. It is declared in the sealed contract itself, at the invariant level, and applied mechanically when the verifier encounters a violation. This means severity is reproducible across runs and across reviewers.

8.5 Full and Lite Verify

Inherited from v1.1 with clarifications: Full VERIFY runs all three layers. Lite VERIFY is permissible for small, low-risk changes and typically runs L1 only, or L1 plus a lightweight L2. The Master Build Guide specifies when Lite is appropriate.

Full VERIFY is mandatory at: milestone closings, contract extensions, any changes touching Critical or High severity invariants, any changes affecting enterprise-governed contracts, and any changes that the team's lead suspects may have broader implications than the change scope suggests.

9. Phase 6 — RECONCILE

RECONCILE is the reconciliation phase — the disciplined sweep that compares artefacts against each other and against reality to detect drift. In v2.0, RECONCILE remains operational at team level (weekly or sprint-end cadence), but also feeds into the enterprise Sealed Review Board's quarterly review in full mode.

9.1 Trigger

RECONCILE is cadence-driven, not event-driven. It runs at the end of every sprint (or week, or milestone — whichever is shortest) regardless of whether drift is suspected. Running RECONCILE only when drift is suspected means missing the drift that has not yet been noticed, which is exactly the drift the phase is designed to catch.

9.2 Primary AI Role

In RECONCILE, the AI acts as an auditor. It compares artefacts mechanically and reports deltas. It does not interpret the deltas — that is the team's job. The AI's value is in thoroughness and consistency, catching drift patterns a human might miss.

9.3 The Five Reconciliation Pairs

Inherited from v1.1 with SEA-native updates, the five pairs are:

Pair	What is compared	Catches
Pair A — Code ↔ Contract	Implementation vs sealed contract specification	Missing operations, signature drift, invariant violations
Pair B — Change History ↔ Code	What the contract's change history declares vs what the repository shows	Ungoverned modifications to sealed contracts
Pair C — DB Schema ↔ Queries	Database column references vs actual schema state	Column rename drift, type drift, missing columns
Pair D — Sealed baseline ↔ current files	Contract file hashes at sealing vs current file hashes	Contract files modified outside the extension protocol
Pair E — Intent ↔ Contract	What the intent document describes vs what the contract currently specifies	Contract evolution that was not reflected back into intent

Pair E is new in v2.0. In v1.1, the intent concept was informal and there was no artefact to reconcile with. In v2.0, the intent document is durable, and when the contract evolves, the intent document must evolve in step. Catching divergence between them is a specific kind of drift that Pair E exists for.

9.4 Outputs

RECONCILE produces:

- A delta report per pair, showing any divergence detected
- Classification of each delta: benign (explained and acceptable), drift incident (requires formal logging), or escalation (requires SRB attention in full mode)

- Updated drift incident log with any new incidents
- Input to the next quarterly SRB review in full mode; summary report for team leadership in team-autonomous mode

9.5 Exit Gate

RECONCILE exits when zero open deltas remain. Every delta is either resolved (code fixed to match contract, or contract extension filed to match code), accepted (documented as a known state to be addressed in a specific future cycle), or escalated (moved to the SRB for decision in full mode).

A RECONCILE phase that exits with unresolved deltas in an 'ignored' state is a RECONCILE that has not done its job. The discipline is to resolve, accept, or escalate — never to silently tolerate.

9.6 Connection to SEA's Quarterly Review

In full mode, each quarterly Sealed Review Board session is informed by the twelve (or thirteen) weekly RECONCILE reports that preceded it. The SRB does not re-run reconciliation; it reviews the accumulated findings from the teams' weekly sweeps, looks for patterns across teams, and makes the portfolio-level decisions that are beyond any individual team's scope.

This structure is why RECONCILE is both a team-level phase and a feeder into enterprise governance. The team catches drift weekly; the SRB catches organisational patterns quarterly. Neither alone is sufficient. Together they produce the early-warning system that keeps sealed architectures from decaying silently.

10. Phase 7 — EVOLVE

EVOLVE is the phase that handles change to sealed contracts. In v2.0, EVOLVE is explicitly the team-level implementation of SEA's three evolution mechanisms: extension, parallel-live replacement, and Major Version Bump. EVOLVE is event-driven — it is invoked when needed, not on a cadence.

10.1 Trigger

EVOLVE is triggered when a new capability is needed that a sealed contract does not support. The trigger is explicit: the team recognises that the work ahead will require a change to a sealed contract, and before any implementation begins, EVOLVE is invoked to determine which evolution mechanism applies and to execute the appropriate protocol.

10.2 Primary AI Role

The AI's role in EVOLVE is extension designer. It proposes how the new capability can be accommodated while preserving the existing sealed contracts. For most cases, this means drafting an extension proposal. For harder cases, it may mean proposing a parallel-live replacement. For the rarest cases, it means recognising that a Major Version Bump is warranted and drafting the proposal for that.

10.3 The Three Mechanisms

EVOLVE in v2.0 selects between three mechanisms based on criteria drawn from SEA 1.0 and the Implementation Handbook:

Mechanism 1 — Extension

Used when the new capability can be added alongside the existing contract without modifying it. The team drafts an extension proposal following the pattern catalogue in Handbook Section 18, submits it through the governance flow (full mode: Contract Owner + Reviewer; team-autonomous mode: team leads), and on approval proceeds to implementation.

Typical duration from proposal to approved extension: 3-7 working days.

Mechanism 2 — Parallel-Live Replacement

Used when the new capability cannot be added alongside — when the existing contract is almost right but needs to be re-shaped — and when active consumers require continuity of service. The team follows the parallel-live playbook in Handbook Section 19, with SRB approval required in full mode.

Typical duration from proposal to completed migration: 6-15 months for enterprise-scale contracts, often shorter for team-scale contracts.

Mechanism 3 — Major Version Bump

Used rarely, when the purpose the contracts serve has fundamentally changed. At team level, this typically corresponds to a product pivot, a regulatory transformation, or the discovery that a set of related contracts were all built on flawed premises that cannot be corrected through extension. The team follows the Major Version Bump protocol in Handbook Section 21, with mandatory executive sponsorship in full mode.

Typical duration: 6-24 months. Rare at team scale.

10.4 Selecting the Right Mechanism

Choosing the right mechanism is an act of judgement, supported by explicit criteria:

Situation	Mechanism
New capability fits within existing extension policy	Extension
New capability would violate extension policy but contract purpose is unchanged	Parallel-live replacement
Contract has accumulated many extensions and is becoming incoherent	Parallel-live replacement (consolidate into v2)
Invariants must be weakened or removed	Parallel-live replacement (old contract remains; new contract has different invariants)
System purpose has fundamentally changed	Major Version Bump
Regulatory regime has transformed requirements	Major Version Bump

10.5 What EVOLVE Does Not Do

EVOLVE does not modify sealed contracts. This is an absolute rule. EVOLVE produces proposals for approved modifications, which are then implemented as new contracts (extension) or new contract versions (parallel-live) or new contract sets (MVB). The original sealed contract is not edited.

Teams new to v2.0 sometimes try to use EVOLVE to make 'small corrections' to sealed contracts. The appropriate response in every case is that if a correction is genuinely needed, it goes through the parallel-live protocol — a new version with the correction — not an edit to the sealed original. The sealed artefact stays intact as a historical reference; the corrected version becomes the active contract.

Sealed means sealed. Evolution happens through new artefacts, not through editing old ones.

— The sealing discipline

PART III

Artefacts

The SEA-native artefacts that v2.0 produces. Replacing v1.1's informal SSOT, Decision Log, and Feature Checklist with SEA-compatible equivalents.

11. The Contract and Intent Pair

The single most important artefact change between v1.1 and v2.0 is the replacement of the SSOT document with a paired sealed contract and intent document. The SSOT in v1.1 was an informal document — usually a large markdown file — that served as the team's reference for what the system was supposed to be. It worked at team scale but did not scale to enterprise or across team boundaries.

The contract and intent pair is the SEA-native realisation of the same intuition, made formal and scalable.

11.1 What the Pair Is

A sealed contract is a YAML file following the schema specified in SEA 1.0 Section 10. An intent document is a Markdown file following the structure specified in SEA 1.0 Section 9.3. The two are always paired — a contract without an intent document is incomplete, and an intent document without a contract is a draft.

Together they capture everything an SSOT captured, but in a form that is structured (contract is machine-readable YAML), narrative (intent is human-readable prose), portable (both are text files independent of any platform), and governed (both are subject to the SEA governance protocol).

11.2 The Division of Labour

What goes in the contract (YAML)	What goes in the intent document (Markdown)
Interface — operations, inputs, outputs, types	Problem statement — what the contract exists to address
Invariants — numbered, with statements	Invariant rationales — why each invariant matters
Extension policy — permitted and forbidden	Rejected alternatives — designs considered and declined
Dependencies on other contracts	Assumptions — what must remain true for the contract to hold
Change history — append-only log	Failure modes protected against
Metadata — Owner, Reviewer, governance body	Related context — links to decisions, incidents, references

11.3 Why Two Artefacts Rather Than One

The division is deliberate. The contract is machine-readable because verifiers need to consume it mechanically. The intent document is narrative because the reasoning that justifies a contract cannot be compressed into YAML fields without losing its meaning. Attempting to force rationales into YAML produces sparse stub content that is worse than either clean YAML or proper prose.

The pairing means that a new engineer or a new AI assistant encountering a sealed contract for the first time has two complementary entry points. The YAML tells them what the contract does. The Markdown tells them why. Together they provide the complete picture that a v1.1 SSOT provided, but in a form that scales across teams, organisations, and decades.

11.4 Migrating from v1.1 SSOT to the Contract+Intent Pair

Teams migrating from v1.1 typically find that their existing SSOT document contains substantial content that maps to both the contract and the intent document. The migration is not a rewrite from scratch; it is a restructuring:

26. The interface definitions in the SSOT become the interface section of the contract
27. The design rules and invariants in the SSOT become the invariants of the contract, with rationales migrated into the intent document
28. The design rationale, alternative approaches discussed, and background context in the SSOT become the intent document's rejected alternatives and problem statement sections
29. Any decision log entries relating to the contract become the contract's change history

A typical SSOT, depending on its size and thoroughness, migrates to a contract + intent pair in one to three engineer-days. Most of the work is restructuring existing content rather than producing new content; the exception is usually the rejected alternatives section, which v1.1 SSOT documents rarely captured explicitly.

12. The Change History

In v1.1, the team maintained a Decision Log — an append-only record of architectural decisions with IDs, dates, rationales, and status. In v2.0, the Decision Log is retained for non-contract decisions, but for decisions that affect sealed contracts, the change history within the sealed contract itself becomes the primary record.

12.1 Structure

The change history is a field within the sealed contract YAML. Each entry records:

- Date of the change
- Action type (sealed, extended, marked-parallel-live, deprecated, retired)
- Actor (the person, body, or governance authority that authorised the change)
- Notes describing what changed and referencing relevant governance records

12.2 Why It Lives in the Contract

The change history is part of the contract because it is the contract's own biography. A contract without its change history is like a bridge without its maintenance records — the structure may still exist, but no one can know what has happened to it, why, or by whose authority.

Storing the change history inside the contract YAML also means that when the contract file is audited, its history is audited alongside it. The two cannot become disconnected; they travel together through retirements, archival, and organisational changes.

12.3 Division Between Change History and Decision Log

Teams in v2.0 maintain two logs: the change history (per-contract, part of the contract) and the team decision log (for decisions not associated with any specific contract).

Type of decision	Where it is logged
A contract is sealed	Change history of the new contract
An extension is approved for a specific contract	Change history of the extended contract
A parallel-live replacement is initiated	Change histories of both contracts (old and new)
A team adopts a new technology choice (not contract-level)	Team decision log
A team restructures its build pipeline	Team decision log
A team decides to adopt a new development tool	Team decision log
A Major Version Bump is initiated across multiple contracts	Change histories of each affected contract, plus team decision log for the cross-contract coordination

13. The Extension Proposal

The Extension Proposal is the artefact through which a team requests a change to a sealed contract. In v1.1, extensions were handled through the Feature Checklist Template, which was a lighter-weight informal artefact. In v2.0, extensions to sealed contracts require the formal Extension Proposal, while non-contract features continue to use ordinary issue tracking.

13.1 When an Extension Proposal Is Required

An Extension Proposal is required whenever:

- The proposed change adds capability to an existing sealed contract
- The proposed change introduces a new sealed contract that extends or depends on existing ones
- The proposed change strengthens an invariant on an existing contract

An Extension Proposal is not required for changes that are entirely within the transient layer. A refactor of internal implementation that leaves the sealed contract unchanged does not require a proposal; it is ordinary engineering work governed by the team's normal code review.

13.2 Required Contents

An Extension Proposal must contain:

30. Contract affected — the `seal_id` of the contract being extended
31. Proposed extension type — additive fields, new operation, invariant strengthening, or wrapper contract (from the pattern catalogue in Handbook Section 18)
32. Proposed changes — the specific additions being proposed, in YAML form matching the contract's schema
33. Policy check — verification that the proposed extension falls within the contract's extension policy
34. Invariant impact — statement of whether invariants are strengthened, unchanged, or (for wrapper contracts) how new invariants relate to existing ones
35. Implementation impact — which implementations will need updating, what the migration path looks like
36. Proposed intent document update — the specific additions to the intent document that this extension warrants

13.3 Review Flow

The review flow depends on mode:

In full mode: the proposal goes to the Contract Owner and the Contract Reviewer. Both must approve. If the extension crosses team boundaries or governance bodies, the proposal is escalated to the SRB.

In team-autonomous mode: the proposal is reviewed within the team. The team lead plus one other senior engineer (acting as proxies for Owner and Reviewer) must approve. For team-autonomous contracts that represent crucial architectural boundaries, the team may choose to invite an external reviewer from outside the team for additional scrutiny.

13.4 Typical Turnaround

An Extension Proposal for a routine additive extension typically turns around in 3-5 working days. More complex extensions — wrapper contracts, cross-contract dependencies, invariant strengthening — may take 7-14 days. Extensions that require escalation to the SRB in full mode are governed by the SRB's quarterly cadence and may take longer.

14. The Drift Incident Log

The Drift Incident Log is retained from v1.1 with significant refinements. It remains the formal record of detected divergences between artefacts or between artefacts and reality. In v2.0, drift incidents are classified using SEA's severity taxonomy and, in full mode, feed into the enterprise drift incident log.

14.1 What Counts as a Drift Incident

A drift incident is any documented case of:

- Divergence between a sealed contract and its implementation (detected in VERIFY, RECONCILE, or production)
- Divergence between two artefacts that should agree (typically surfaced by a Reconciliation Pair)
- An ungoverned modification to a sealed artefact (typically surfaced by Pair D, the hash check)
- A production issue traced to a contract violation
- An audit finding that surfaces a governance gap

Routine code defects — bugs, logic errors, UI issues — are not drift incidents. They are ordinary engineering work. A drift incident is specifically about the integrity of the sealed architecture itself.

14.2 Record Structure

Each incident record contains the elements specified in Handbook Section 11.4, adapted for team-level use:

```
# Drift Incident – CD-series log entry

id: CD-2026-017
date_detected: 2026-04-22
date_occurred: 2026-04-15 # may be same as detected

detection_mode: reconciliation-pair-D
# (or: verification-failure, production-incident,
#      audit-review, external-report)

affected_contracts:
- auth.user_session.v1

severity: High
severity_rationale: |
  Invariant INV-2 (atomic revocation) was violated in a specific
  code path discovered during weekly reconciliation. No production
  impact yet, but the path is reachable under specific error
  conditions.

root_cause: governance-gap
# (or: tooling-gap, education-gap, deliberate-bypass)
```

```
investigation_findings: |
  The recent extension for bulk revocation was implemented
  correctly for the happy path but did not handle the case where
  one of the underlying single revocations fails partway. The
  extension proposal mentioned atomic behaviour but did not
  explicitly specify handling of this case.

remediation_applied: |
  Implementation updated to handle partial failure by rolling
  back any successful revocations in the batch. Test added to
  exercise the failure path.

preventive_action: |
  Extension proposal template updated to include explicit
  question: 'how does this extension interact with each
  existing invariant under failure conditions?'

status: resolved
```

14.3 Feeding Into Enterprise Governance

In full mode, the team's drift incident log is not isolated. At each quarterly Sealed Review Board session, all team-level incidents from the quarter are aggregated into the enterprise log. The SRB reviews patterns: which contracts are breached most often, which root causes recur, which severity levels are trending up or down.

This feeding is not automatic but it is structural. A team operating in full mode should expect its drift incidents to be visible to enterprise governance, and should record them with that audience in mind. Honest reporting — severity set correctly, root cause classified accurately, remediation documented fully — is how the team contributes to the organisation's overall architectural learning.

14.4 In Team-Autonomous Mode

In team-autonomous mode, the drift incident log is team-local. There is no enterprise log to feed into. But the same discipline applies: incidents are recorded formally, severity is classified using the SEA taxonomy, root causes are investigated honestly, and preventive actions are taken.

A team operating in team-autonomous mode that later transitions to full mode will find its drift incident log ready to be absorbed into the enterprise log without rework. The artefacts were SEA-shaped from the start; they simply change governance context rather than format.

PART IV

Team-to-Enterprise Interface

How team-level work connects to enterprise governance under SEA. Full mode and team-autonomous mode detailed.

15. Proposing Contracts Upward

In full mode, the team does not seal contracts in isolation. Contract candidates identified during FRAME, developed during SEAL, and submitted through the enterprise governance process become part of the organisation's sealed portfolio. This section specifies how that upward flow works.

15.1 The Proposal Moment

A team that has developed a contract candidate to the point of being ready to seal does not simply seal it and announce the fact. The candidate is proposed upward, typically through the team's architect liaison or directly to the governance body that has jurisdiction over the contract's domain.

The proposal consists of:

- The draft sealed contract YAML
- The draft intent document
- An explanation of why the team is proposing this contract as a durable boundary
- A consumer analysis — who will depend on this contract, with which initial implementations
- A proposed Contract Owner and Reviewer, with justification

15.2 The Review

The SRB (or delegated governance body) reviews the proposal with four main questions in mind:

37. Is this contract necessary at the enterprise level, or should it remain team-internal? Some boundaries are genuinely team-scoped; sealing them at enterprise level creates coordination overhead without benefit.
38. Does this contract overlap with existing enterprise contracts? A new contract that duplicates or fragments an existing portfolio area should be reshaped or absorbed, not added as-is.
39. Is the proposed Owner and Reviewer appropriate? The SRB may accept the team's proposal or suggest alternatives — for example, a cross-team Owner for a contract that affects multiple teams.
40. Is the intent document complete and defensible? The SRB is the enterprise's institutional memory for architectural reasoning; contracts entering the portfolio with weak intent documents weaken the portfolio as a whole.

15.3 The Four Possible Outcomes

Outcome	Meaning
Accepted as-is	The contract enters the enterprise portfolio with the team's proposed shape, owner, and reviewer. Rare; most contracts benefit from at least some adjustment.
Accepted with modifications	The contract is welcomed but specific adjustments are requested before sealing — typically clarifications to invariants, adjustments to the extension policy, or owner reassignment.
Returned for rework	The proposal has structural issues that require more fundamental redesign. The team returns to SEAL with specific guidance from the SRB.
Redirected	The SRB identifies that what the team needs is actually an extension of

Outcome	Meaning
	an existing contract, not a new contract. The team returns to SPEC with an extension proposal instead.

15.4 Typical Timeline

A well-prepared contract proposal typically receives an outcome within 2-4 weeks. Complex proposals — especially those affecting multiple teams or regulatory domains — may take longer. Proposals that are returned for rework can add another cycle of similar duration. Teams planning around sealing a new contract should budget 4-8 weeks between 'team is ready to seal' and 'contract is sealed at enterprise level' in full mode.

This timeline is the price of enterprise governance. Teams that find it prohibitive for their context should consider team-autonomous mode rather than attempting to force enterprise-scale processes onto team-scale work.

16. Extension Reviews That Span Scales

Extensions to sealed contracts are the most frequent governance interaction in v2.0. Every additive change, every new capability, every invariant strengthening goes through some form of extension review. In full mode, many of these extensions cross team boundaries — the team proposing the extension is not the same as the team whose Contract Owner must approve it.

16.1 The Cross-Scale Flow

A typical extension flow in full mode:

41. The proposing team identifies a need during SPEC that requires extending an existing sealed contract
42. The proposing team drafts the extension proposal using the template specified in the Implementation Handbook
43. The proposal is sent to the Contract Owner of the contract being extended, who may be in a different team, business unit, or even geography
44. The Contract Owner reviews the proposal against the intent document of the contract — does the extension align with the contract's original purpose?
45. The Contract Reviewer reviews the proposal against technical constraints — is it feasible, does it conflict with other extensions, does it respect the extension policy?
46. Both approve, return for rework, or reject. The proposing team receives the decision and proceeds (or not) with implementation

16.2 Communication Patterns

When extensions cross team boundaries, communication discipline matters. Three patterns that work:

- Asynchronous first — the proposal is the primary artefact; synchronous meetings are only used when asynchronous review surfaces specific questions
- Explicit acknowledgement — when the proposal is received, the Contract Owner acknowledges within one working day, even if the full review will take longer
- Escalation channels — if an extension review stalls beyond the team's agreed turnaround norms, there is a named escalation path, typically through the SRB

16.3 Resolving Disagreements

Sometimes the proposing team and the Contract Owner disagree on whether an extension is appropriate. Four patterns help resolve such disagreements:

Disagreement type	Resolution approach
Proposing team thinks extension is within policy; Owner thinks it is not	Consult the actual extension policy in the contract YAML. If ambiguous, the policy itself may need clarification (a small extension to the policy, not the contract).
Proposing team thinks an invariant should be weakened; Owner disagrees	Weakening is typically forbidden by policy. The correct path is parallel-live replacement, not extension.
Owner thinks the team should restructure rather than extend	Owner's position usually wins — Owners have broader portfolio context. But team should document its alternative for the record.

Disagreement type	Resolution approach
Both agree on intent but disagree on shape	Small number of alternatives, each with an assessment. Owner decides; team defers.

16.4 Escalation to SRB

Extensions should rarely require SRB involvement. When they do, it is typically because:

- The extension has implications across multiple contracts and Owners
- The extension raises policy questions that will affect future extensions
- The Owner or Reviewer is unavailable and succession has not been formally arranged
- The disagreement is between teams that cannot resolve it bilaterally

SRB escalation is not a punishment. It is the mechanism for ensuring that cross-cutting concerns are addressed at the right scale. Teams that escalate appropriately are doing governance correctly. Teams that never escalate may be either exceptionally well-aligned or papering over genuine disagreements.

17. The RECONCILE → Quarterly Review Bridge

In v1.1, RECONCILE was a weekly discipline that caught drift at team scale. In v2.0 full mode, the weekly rhythm remains, but each quarterly cycle produces an input into the SRB's quarterly review. This creates a two-tier structure: teams catch drift as it happens, and the SRB catches organisational patterns over longer windows.

17.1 The Weekly Team Rhythm

At the team level, RECONCILE runs on whatever short cadence the team operates — typically weekly for continuous-flow teams, sprint-boundary for scrum teams, milestone-boundary for waterfall-flavoured teams. The shorter the better; weekly is usually optimal. The discipline is that the phase runs regardless of whether drift is suspected.

Each weekly RECONCILE produces:

- A delta report across the five reconciliation pairs
- Any new drift incidents, logged and investigated
- Any escalations needed (contracts with unresolvable deltas that require SRB attention)
- Input to the team's contract integrity scores (Handbook Section 2)

17.2 The Quarterly SRB Review

At the SRB level, the quarterly review takes the twelve (or thirteen) weekly reports from each team in the body's jurisdiction and reviews them as aggregate data. The SRB is not re-running reconciliation; it is looking for patterns across time and across teams.

Specifically, the SRB asks:

- Which contracts are drift-prone? Contracts that produce drift incidents repeatedly may be poorly specified or serve an unclear purpose.
- Which root causes recur? If most drift is caused by tooling gaps, tooling investment is the priority. If it is governance gaps, governance tightening is needed.
- How are contract integrity scores trending? A portfolio-level decline is more urgent than any individual incident.
- Are succession events being handled well? Contracts whose integrity scores drop after ownership transitions reveal weakness in the succession protocol.
- Are any contracts ready for retirement? Zero-consumer contracts should be surfaced and retired formally rather than left to ghost-ownership.

17.3 The Feedback Loop

The SRB's quarterly findings flow back to the teams:

- Contracts identified as drift-prone get specific attention — either renewed governance, re-specification, or candidacy for parallel-live replacement
- Root-cause patterns produce organisation-level interventions — new training, new tooling, updated policies
- Integrity score declines get escalated; teams are expected to produce remediation plans within 30 days
- Successful patterns are surfaced as examples for other teams to learn from

This feedback loop is what makes SEA adoption a living system rather than a static framework. Teams and the SRB learn from each other quarter by quarter. The architecture improves not through any single big intervention but through the accumulated minor adjustments that a good feedback loop produces.

Team rhythms catch drift weekly. Enterprise rhythms catch patterns quarterly. The feedback between them is what keeps the architecture alive.

— The two-tier rhythm

18. Operating in Team-Autonomous Mode

Team-autonomous mode is the operating context for teams using SDLC v2.0 in organisations that have not adopted SEA at enterprise level. It is not a compromised or degraded mode; it is a legitimate way to operate v2.0. This section specifies what team-autonomous mode looks like in practice.

18.1 What Is the Same

Most of v2.0 is unchanged in team-autonomous mode:

- The seven phases operate identically — same triggers, same outputs, same exit gates
- The artefacts are the same — sealed contracts, intent documents, extension proposals, drift incident logs
- The compliance state vocabulary and severity classification are the same
- The three evolution mechanisms (extension, parallel-live replacement, Major Version Bump) operate the same way
- Source-First, one-chat-per-unit, cross-family review — all the same

18.2 What Is Different

What changes is where the governance authority resides:

Governance role	Full mode	Team-autonomous mode
Contract Owner	Named individual, potentially outside the team, with formal succession protocols	Senior team member, with team-local succession when they leave
Contract Reviewer	Named individual, different from Owner, formally accountable	Another senior team member, rotating with Owner
Sealed Review Board	Standing enterprise body, 4-8 members, quarterly cadence	The team itself, meeting on quarterly rhythm for formal review
Extension approval	Contract Owner + Reviewer, possibly cross-team	Team lead + senior engineer, within the team
Parallel-Live approval	SRB	Team leadership with senior technical stakeholder
Major Version Bump approval	SRB unanimous + executive sponsor	Team leadership + team's senior technical stakeholder with explicit accountability
Drift incident log	Feeds into enterprise log	Team-local, preserved for eventual enterprise integration
Verifier tooling	Enterprise platform team	Team-built or off-the-shelf, maintained within the team

18.3 The Role of the Senior Technical Stakeholder

Team-autonomous mode requires at least one senior technical stakeholder outside the team who is accountable for its architectural decisions — even if no enterprise SRB exists. This person might be a CTO, a VP of Engineering, a principal architect, or a technical co-founder. Their role is to:

- Witness Major Version Bumps when they occur
- Receive escalations when the team's internal governance cannot resolve disagreements
- Provide external perspective on annual reviews of the team's sealed portfolio
- Serve as the bridge to enterprise governance if the organisation later adopts SEA

Without such a person, team-autonomous mode is at risk of insularity — the team becomes its own only reviewer, and the discipline that SEA's governance structure provides through external accountability is lost. A team without an external stakeholder can still operate v2.0, but it should recognise this as a gap and seek to close it.

18.4 Preparing for Possible Transition to Full Mode

A team operating in team-autonomous mode that anticipates its organisation may eventually adopt SEA can take steps now to ease the future transition:

- Keep artefacts in SEA-canonical format from the start — YAML contracts, Markdown intent documents, proper schema
- Maintain a proper drift incident log with severity classification, not informal notes
- Document succession even though it is informal — when an Owner or Reviewer role transitions, record it
- Treat the team's annual external review as practice for quarterly SRB reviews

A team that has operated in team-autonomous mode with this discipline is typically the easiest to bring into full mode when the organisation adopts SEA. The artefacts are ready; the culture is ready; only the governance context needs to shift.

PART V

Migration from v1.1

Who should migrate, who should not. How to migrate when it is time. What to preserve, what to transform.

19. Who Should Migrate, Who Should Not

Version 1.1 of the AI-Native SDLC is not deprecated or retired. It remains in its archived form, specification readable, governance protocol intact. Teams currently operating on v1.1 are not obligated to migrate. In many cases, they should not.

This section provides honest guidance on who benefits from migrating and who should stay on v1.1.

19.1 Who Should Migrate

Teams in any of these situations will meaningfully benefit from v2.0:

- The team's organisation has adopted SEA or is in active SEA adoption — v2.0 is designed for this context and v1.1 will increasingly feel misaligned as SEA governance operates around a team still using v1.1 vocabulary
- The team has grown past 15 engineers — v1.1's informal artefacts become harder to maintain at this scale; v2.0's formal artefacts scale more gracefully
- The team is entering regulated territory — healthcare data, financial services, critical infrastructure — where formal artefact governance is becoming a compliance expectation
- The team has experienced drift incidents that traced to governance gaps — v2.0 addresses these gaps more directly than v1.1
- The team is planning to open its architectural artefacts to external partners, regulators, or auditors — v2.0's SEA-native artefacts are more portable and auditable

19.2 Who Should Not Migrate

Teams in any of these situations should stay on v1.1:

- Small team (under 5 engineers), short product horizon (under 2 years expected lifetime) — v2.0's overhead is not justified
- Organisation has no intention of adopting SEA and the team is not positioned to lead the adoption — v2.0 in team-autonomous mode works, but requires senior technical stakeholder support that may not exist
- Team is in the middle of a production crisis — migration is not a crisis activity; stabilise first, migrate later
- Team is mid-way through a major v1.1 implementation (a large SSOT, deep Decision Log) — complete the work on v1.1, then consider migration in the next product cycle
- The team's people are fatigued by process changes — v2.0 is a substantial shift; imposing it on a change-fatigued team produces theatre, not discipline

19.3 The Non-Migration Path

A team choosing to stay on v1.1 is making a legitimate choice. Nothing about the existence of v2.0 invalidates v1.1. The archive preserves v1.1 in full; its specification is readable; its rituals are operational.

What does change for v1.1 teams is that they should be aware:

- New methodology development will focus on v2.0 — new guides, templates, and operational improvements will target v2.0 first
- When the team eventually migrates (if it does), the migration will likely be longer the further v1.1 practice has evolved on its own

- Integration with SEA-adopting organisations will require either migration or translation, which has a cost

None of these consequences are urgent. A v1.1 team that expects to continue operating v1.1 for two or three more years will be fine.

20. The Migration Path

For teams that have decided to migrate from v1.1 to v2.0, this section specifies the path. The migration is typically one to two sprints in duration, depending on the size of the team's existing v1.1 artefacts and the team's familiarity with SEA vocabulary.

20.1 Pre-Migration Preparation

Before beginning the migration, the team should:

47. Read SEA 1.0 — at minimum Parts I, II, and III. The team must internalise the SEA philosophy before adopting the SEA-native SDLC.
48. Read the SEA Implementation Handbook Volume 1 — particularly Parts II (enforcement) and IV (evolution). The Handbook specifies the operational mechanics that v2.0 builds on.
49. Read this SDLC v2.0 document in full.
50. Identify which mode (full or team-autonomous) applies to the team's context.
51. Identify a migration lead — one person on the team responsible for driving the migration through to completion.

20.2 The Migration Sprint

The core migration work typically happens in a focused sprint:

Day 1-2: Artefact inventory

List all v1.1 artefacts currently in use: SSOT documents, Decision Log, Feature Checklists, drift incident log, any contracts defined informally. For each artefact, note its current size, last update, and the team members who maintain it.

Day 3-5: SSOT migration

The SSOT is the largest artefact to migrate. For each interface or module currently described in the SSOT, decide whether it is genuinely durable (warranting sealing) or transient (remaining informal). For each durable element, produce a sealed contract YAML and a paired intent document, following the schemas in SEA 1.0.

This is the most substantive part of the migration. Expect to discover that some v1.1 SSOT content does not cleanly map to either a contract or intent — typically this is implementation-specific content that was mixed into the SSOT informally. Such content either moves into ordinary engineering documentation or is discarded.

Day 6-7: Decision Log migration

For each entry in the Decision Log, determine whether it relates to a specific sealed contract. Entries that do migrate into the change history of that contract. Entries that do not — technology choices, tooling decisions, process changes — remain in a team decision log, which continues to exist alongside the sealed contracts.

Day 8: Verification strategy

For each sealed contract, define the verification strategy: which invariants are checked statically, which at runtime, which through manual review. Update the verifier tooling (or create it, if the team is setting it up for the first time) to run these checks.

Day 9: Governance setup

Identify Contract Owners and Reviewers for each sealed contract. In full mode, coordinate with the SRB for any contracts being proposed for enterprise adoption. In team-autonomous mode, confirm the team-internal roles and the external stakeholder who will witness major events.

Day 10: Transition cutover

Announce the migration completion to the team. From this point forward, work operates on v2.0. The v1.1 SSOT is archived (not deleted). The Decision Log is split between contract change histories and the ongoing team decision log. New work follows v2.0 phases.

20.3 The First Quarter After Migration

The first quarter after migration is where the team's new habits are tested against real work. Three common patterns:

- Teams occasionally default back to v1.1 habits when under deadline pressure — the migration lead watches for this and gently redirects
- Teams discover that some contracts were sealed too eagerly or too cautiously during migration — this is normal and addressed through ordinary evolution mechanisms (extension, or if needed, parallel-live replacement)
- Teams find that the formal extension protocol takes longer than the v1.1 informal process — this is expected; the benefit is in caught drift and cleaner governance over time, not in faster individual changes

20.4 Marker of Successful Migration

A team has successfully migrated from v1.1 to v2.0 when three conditions hold:

52. Every active sealed contract has a complete intent document, current Owner and Reviewer, and working verifier coverage
53. Every new change to a sealed contract follows the extension protocol, without exception
54. The first post-migration RECONCILE sweep produces a delta report using the v2.0 five pairs and compliance state vocabulary, not informal v1.1 language

Teams that achieve these three conditions within one quarter are well-adopted. Teams that take longer are still on the path; the migration is not a fail-fast process, it is a steady internalisation of new habits.

ANNEXES

Supporting Material

Annex A — CD1–CD4 Reframed Through SEA

The AI-Native SDLC v1.1 drew its authority from four documented drift incidents observed during a real production build: CD1 (Sealed Function Count Drift), CD2 (Regeneration Loss), CD3 (Wrong File Attribution), and CD4 (Status Drift on Vulnerability Fixes). These incidents remain authoritative evidence for the framework.

Under v2.0's SEA-native view, each incident can be reframed not just as a process gap but as a specific failure of one or more of SEA's principles. This reframing does not change what happened; it changes how the incidents are understood, and how they inform the protections that v2.0 offers.

A.1 CD1 — Sealed Function Count Drift

In the original framing, the SSOT declared 19 sealed functions while the code had 23. Different AI sessions produced different counts. Neither was wrong when written; the delta accumulated unnoticed.

In SEA-native framing, this is a failure of Principle 1 (Contracts are durable; implementations are transient). The SSOT was attempting to function as a contract but was not sealed — it was a working document that different sessions could edit informally. A proper sealed contract with hash verification would have made the divergence impossible: any implementation diverging from the contract would have been caught by the verifier before merge.

v2.0 addresses this directly. Pair A (Code ↔ Contract) during RECONCILE catches exactly this pattern; Pair D (Sealed baseline ↔ current files) catches it even earlier, by detecting any contract file modification outside the extension protocol.

A.2 CD2 — Regeneration Loss

In the original framing, the SSOT was 'updated' by regenerating it. Two sections silently disappeared. The AI did not know which sections to preserve. The loss was caught only by a table-of-contents hash change.

In SEA-native framing, this is a failure of Principle 2 (Intent is captured, not just behaviour). The SSOT was prose that mixed intent with specification, with no clear boundary between the two. When regenerated, the AI had no way to know which prose was load-bearing and which was scaffolding. A proper intent document with explicit required sections (problem statement, invariants with rationale, rejected alternatives, assumptions, failure modes) would have made regeneration safer: the structure itself specifies what must be preserved.

v2.0 addresses this by using structured intent documents where the sections are required. A regeneration that loses the rejected alternatives section cannot produce a valid intent document; the missing section is immediately visible. Pair E (Intent ↔ Contract) during RECONCILE also catches divergence between the intent document and the contract it is paired with.

A.3 CD3 — Wrong File Attribution

In the original framing, a bug fix was applied to MivaReelPlayer.kt. The bug persisted because the logic actually lived in ChapterPlayerScreen.kt — a file the AI had never opened in that chat. It described code instead of reading it.

In SEA-native framing, this is a failure of Principle 7 (AI-readability is structural) combined with the Source-First rule. The AI hallucinated code based on its prior training rather than reading the actual source. A contract that explicitly specifies where its implementations live, and an environment that requires opening files before describing them, would have prevented the error.

v2.0 retains Source-First as an operational rule in BUILD. The structural improvement is that sealed contracts specify their own implementation locations, so that AI assistants working against the contract always know where the actual code lives — no hallucination required.

A.4 CD4 — Status Drift on Vulnerability Fixes

In the original framing, V1–V11 vulnerability fixes were applied to code. The SSOT still showed them as 'Pending.' Both were correct when written. They were mutually inconsistent for weeks, caught only during a release readiness review.

In SEA-native framing, this is a failure of Principle 3 (Succession is designed, not assumed) and of the Authority Chain. The SSOT was authored by one AI session; the code changes were authored by another; neither knew about the other. No single artefact held the authoritative status, and no governance process reconciled them. Over time, the two drifted.

v2.0 addresses this through the Authority Chain (DATABASE > DECISION LOG > SSOT > CODE > DOCS) made operational through the contract's change history. Every change to a sealed contract is recorded in the change history, which is then the single authority. Other artefacts — including any SSOT-like documents — are derived views that must reconcile to the change history. Pair B (Change History ↔ Code) during RECONCILE catches any ungoverned divergence.

A.5 What the Reframing Tells Us

Each of the four incidents that produced v1.1 was a specific failure of a specific SEA principle. This is not accidental. The principles were derived, in retrospect, from the patterns that the incidents revealed. v1.1 was groping toward SEA without having articulated it; SEA 1.0 articulated it explicitly; v2.0 now operationalises it fully.

This is how methodology matures. A first-generation framework identifies what breaks. A second-generation framework names the principles those breaks violated. A third-generation framework builds protections that prevent the specific violations. v2.0 is the third generation of the same learning.

Annex B — Terminology Mapping v1.1 → v2.0

This annex provides a direct mapping of v1.1 terms to v2.0 terms, with rationale for each change. Teams migrating can use this as a quick reference.

v1.1 term	v2.0 term	Why the change
SSOT (Single Source of Truth)	Sealed contract + intent document pair	SEA-native artefacts are structured and portable; the pair replaces the informal monolithic document
Decision Log	Change history (for contract decisions) + team decision log (for other decisions)	Contract-related decisions belong inside the contract; other decisions remain in a team log
Feature Checklist Template	Extension proposal (for contract-affecting features) + ordinary issue tracking (for other features)	Contract changes require formal proposals; other features use normal tooling
L1 / L2 / L3 VERIFY tiers	Five compliance states + severity classification	Richer vocabulary; better decision guidance; SEA-aligned
Sealed interface	Sealed contract	The broader SEA term; interfaces are one kind of sealed element among several
Anti-drift-by-design	SEA-native governance	The principle is the same; v2.0 makes its mechanism explicit
Sealed Eternal API Layer (SEAL)	Sealed Enterprise Architecture (SEA)	The specific Android implementation was called SEAL; the architectural philosophy is SEA
PL-01 through PL-09 prompt library	Prompts retained; aligned to SEA-native artefacts in the Master Build Guide (future v1.2)	Operational rather than conceptual; the prompts need minor updates to align vocabulary
CD-series drift incidents	Drift incidents with SEA severity classification	The CD numbering convention continues; classification is richer
Full vs Lite VERIFY	Retained, unchanged	Operational distinction that works well and did not need SEA-native rework

The terminology shifts are not cosmetic. They reflect substantive changes in what the artefacts are and how they are governed. Teams migrating should not simply search-and-replace v1.1 terms with v2.0 terms in their existing documentation; they should understand what has changed and re-express the content in the new forms.

Annex C — Version Relationship Record

This annex documents the formal relationship between v1.1 and v2.0, following the principles SEA itself specifies for sealed artefact version relationships.

C.1 Version Status

Version	Status	Purpose
v1.0	Retired	Initial public release; superseded by v1.1
v1.1	Archived	Final version of the pre-SEA SDLC; remains readable, no longer the current framework
v2.0	Current (sealed)	The SEA-native AI-Native SDLC; current framework as of April 2026

C.2 What Archival Means

v1.1 is archived, not deleted. This means:

- The v1.1 specification remains readable in its original form
- Teams currently operating on v1.1 can continue to do so indefinitely
- v1.1 is not updated — its archived state is final
- References to v1.1 in external documentation remain valid; the version number points to a stable specification
- New methodology development focuses on v2.0

This is the treatment SEA specifies for retired sealed artefacts (Handbook Section 22). It is how v2.0 demonstrates the principle it teaches — that versioned artefacts end with dignity, their history preserved, not quietly deleted when they are no longer current.

C.3 What Triggered the Major Version Bump

The Major Version Bump from v1.1 to v2.0 was triggered by one of the conditions SEA 1.0 Section 21 specifies: the discovery that the premises the previous version rested on had been superseded by a broader framework.

Specifically, v1.1 treated the sealed architecture informally, as a set of principles to be held through team discipline. SEA 1.0 articulated the full architectural philosophy those principles implied, including governance structures, enforcement mechanisms, and evolution protocols that v1.1 did not specify. With SEA 1.0 published, v1.1's treatment became inadequate not because it was wrong but because it was incomplete. A new major version was warranted to bring the SDLC into alignment with its now-articulated foundation.

C.4 Forward Compatibility

Teams that migrate from v1.1 to v2.0 using the path in Section 20 do not lose historical artefacts. The v1.1 SSOTs become v2.0 contract+intent pairs with their content restructured, not rewritten. The v1.1 Decision Log entries become v2.0 change history entries where applicable. The v1.1 CD-series drift incidents continue to exist with the same identifiers, classified under v2.0 severity.

This forward compatibility is intentional. SEA teaches that major version bumps should preserve the work of previous versions through transformation rather than destruction. v2.0's migration path demonstrates this at the methodology level: what v1.1 produced continues to live in v2.0, in a different form.

C.5 Future Versions

v2.0 is current. It is not expected to be replaced quickly. Based on the framing in SEA 1.0 Section 21, the next Major Version Bump — v3.0, if it ever exists — would be triggered only by one of:

- A fundamental shift in what AI-assisted development requires (e.g. AI models reaching a level of capability where the current phase structure no longer maps to the work)
- A substantial evolution of SEA itself (SEA 2.0 or later) that reshapes the foundation this SDLC rests on
- Accumulated empirical evidence from adoption that reveals structural flaws in v2.0 that cannot be addressed through extension

None of these conditions is anticipated in the near term. v2.0 is expected to be the current SDLC for several years.

— End of Document —

The AI-Native SDLC v2.0 · Team-Level Operations Under Sealed Enterprise Architecture
Suneesh Sidharthan · suneesh.sidharthan80@gmail.com · +91 9986898076
Released under CC BY 4.0 · April 2026